Théorique (concept) - Le cycle complet de la réalisation du produit

Formation SDLC — préalable théorique aux ateliers

Cette page est le **parcours de formation théorique** à compléter avant de participer aux ateliers SDLC. Elle vise à développer une compréhension commune du cycle, des spécifications, des modèles, des preuves attendues, de la traçabilité, de la gouvernance Jira/Xray, de la modélisation comportementale et des mécanismes de validation des acquis.

> **Distinction importante :** cette page ne sert pas à documenter un atelier réel. Les cas concrets, notes d'atelier, décisions, assignations, owners, risques, actions et suivis doivent être consignés dans la page Pratique atelier — Le cycle complet de la réalisation du produit. La présente page reste centrée sur l'apprentissage théorique, les questions de compréhension et les examens.

Objectifs d'apprentissage

- Comprendre pourquoi le SDLC existe comme boucle de réduction de risque et d'apprentissage, et non comme simple procédure administrative.
- Situer les phases ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER et CONFIRMER dans le cycle complet.
- Maîtriser la découverte, la compréhension, l'écriture et la catégorisation des spécifications IF... THEN....
- Comprendre le rôle des modèles, diagrammes, mockups, designs et modèles comportementaux comme représentations complémentaires des spécifications.
- Comprendre comment différentes familles d'outils de modélisation comportementale, MBT, automatisation et documentation testable peuvent soutenir une documentation produit vivante lorsqu'elles sont utilisées pour répondre au besoin : standardiser la livraison, relier spécifications, modèles, preuves, automatisation et apprentissage.
- Comprendre comment Xray, test cases, test sets et test coverage soutiennent la documentation produit testable et le scope des Jira.
- Comprendre les principes de gouvernance Jira : même projet produit, hiérarchie EPIC → FEATURE → STORY / BUG, petites releases, tableaux SDLC produit et tableaux d'équipes.
- Comprendre comment les critères qualité, la qualimétrie, l'automatisation et l'IA dépendent de spécifications, modèles et sources de vérité fiables.
- Savoir utiliser les références officielles : SDLC officiel, pages « Comment... », SPÉCIFICATION, Recommandations Jira/Xray, Contrat qualité, Qualimétrie et Lexique.

Parcours progressif de la formation

Étape	But	Contenu théorique	Validation
0. Diagnostic initial	Établir le point de départ du participant.	Questionnaire rapide sur SDLC, phases, rôles, risques et réflexes de base.	QCM non bloquant.
1. Fondements SDLC et posture qualité	Comprendre la logique globale du cycle.	SENSE / FORGE / CARE, Continuous Specification, portes, critères de sortie, rôles, contrat qualité.	Quiz bloc 1.
2. Spécifications comme langage commun	Comprendre ce qu'est une spécification et pourquoi elle structure le SDLC.	Définition, IF... THEN..., atomicité, observabilité, traçabilité, preuve attendue, cas limites.	Quiz bloc 2.
3. Catégorisation et stratégie de validation	Savoir classifier une spécification pour savoir comment la prouver.	Niveau d'intégration, interface observée, angle d'observation, type de validation.	Quiz bloc 3.
4. Modélisation produit, mockups, designs et MBT	Comprendre la concordance entre spécification textuelle, représentation visuelle et modèle comportemental exécutable.	Flux, états, séquences, modèles de domaine, mockups, designs UI/UX, MBT et familles d'outils possibles pour relier modélisation, documentation produit, automatisation, IA et validation.	Quiz bloc 4.
5. Xray, test sets et couverture	Comprendre la documentation produit testable.	Test case comme spécification, test sets	Quiz bloc 5.

		par périmètre, test coverage dans Jira.	
6. Gouvernance Jira et découpage de livraison	Comprendre comment structurer la livraison sans explosion d'outils et de tickets.	Même projet produit, EPIC → FEATURE → STORY / BUG, petites releases, tableaux produit et équipes.	Quiz bloc 6.
7. Qualimétrie, automatisation et IA	Comprendre comment mesurer, automatiser et utiliser l'IA sans amplifier le flou.	Métriques, signaux, preuves, CI/CD, automatisation progressive, IA assistée par sources fiables.	Quiz bloc 7.
8. Intégration dans les phases SDLC	Relier les acquis théoriques aux phases ÉLICITER → CONFIRMER.	Ce que chaque phase attend des spécifications, modèles, preuves, couvertures et signaux.	Examen sommaire.
9. Validation globale	Vérifier la compréhension complète avant la pratique.	Cas transversal théorique couvrant signal, spécification, modèle, Xray, Jira, validation, couverture et retour terrain.	Examen récapitulatif.
10. Passage vers la pratique	Appliquer les acquis dans un atelier réel.	Utilisation de la page pratique dédiée.	Observation ou rétroaction en atelier.

Étape 0 — Diagnostic initial

Consigne : choisir la meilleure réponse. Le diagnostic sert à orienter la formation; il n'est pas bloquant.

1. **Quel est le bon premier réflexe SDLC lorsqu'un signal client ou terrain arrive, mais que la cause et l'impact ne sont pas encore confirmés ?**
   a. PLANIFIER, pour réserver de la capacité rapidement.

b. ÉLICITER, pour qualifier le signal, la source, l'impact, les faits et les hypothèses.
c. IMPLÉMENTER, parce qu'un développeur peut probablement corriger.
d. DIFFUSER, parce que le client attend une solution.

2. **Comment faut-il traiter une cause proposée par un participant lorsqu'elle n'est pas encore prouvée ?**
   a. Comme un fait confirmé.
   b. Comme une hypothèse à documenter et valider.
   c. Comme une décision produit.
   d. Comme une preuve de validation.
3. **À quoi sert principalement un RACI dans le contexte SDLC ?**
   a. Remplacer la discussion de workshop.
   b. Déterminer qui réalise, approuve, est consulté ou informé.
   c. Fournir toutes les réponses techniques.
   d. Créer automatiquement les tickets Jira.
4. **Que faire si une spécification peut être interprétée différemment par le PO, le développement, l'IV ou le support ?**
   a. Continuer vers l'implémentation et clarifier plus tard.
   b. Demander au développeur de choisir l'interprétation la plus simple.
   c. Supprimer la spécification pour éviter le débat.
   d. Revenir à DÉFINIR pour clarifier le comportement observable et les preuves attendues.
5. **Quel est le rôle principal d'un atelier SDLC bien mené ?**
   a. Informer passivement les participants.
   b. Réduire le risque par confrontation de perspectives, clarification et décision.
   c. Prendre des notes sans décider.
   d. Remplacer la documentation officielle.

Étape 1 — Fondements SDLC et posture qualité

Notions à comprendre

Fondement	À comprendre	Références
Pourquoi les ateliers existent	Un atelier SDLC n'est pas une réunion d'information : c'est un mécanisme de réduction de	SDLC officiel RACI officiel

	risque par confrontation de perspectives.	
SENSE / FORGE / CARE	SENSE clarifie, FORGE réalise, CARE observe, supporte et alimente l'amélioration continue.	SDLC officiel Lexique
Continuous Specification avant CI/CD	La qualité se sécurise surtout par la clarification, les spécifications, les modèles, les risques, les preuves attendues et les critères de sortie.	SPÉCIFICATION SDLC officiel
Portes et critères de sortie	Si les critères d'une phase ne sont pas satisfaits, le bon réflexe est de rendre l'écart visible et de revenir à la bonne phase.	SDLC officiel Contrat qualité
Posture qualité	La qualité n'est pas un rôle isolé en fin de chaîne : elle est portée par chaque rôle à travers les décisions, les preuves, les critères de sortie et les apprentissages.	Contrat qualité RACI officiel

Quiz bloc 1 — Fondements SDLC

1. **Pourquoi le SDLC distingue-t-il SENSE, FORGE et CARE ?**
   a. Pour séparer artificiellement les équipes.
   b. Pour éviter les responsabilités partagées.
   c. Pour comprendre, réaliser, puis observer et apprendre dans une boucle complète.
   d. Pour remplacer le RACI.
2. **Quel est le risque principal lorsqu'une équipe saute trop vite d'un signal ou d'un besoin vers l'implémentation ?**
   a. Aller plus vite que prévu.
   b. Avoir trop de documentation officielle.
   c. Transférer des ambiguïtés vers les phases suivantes.

d. Consulter trop de rôles.

3. **Que doit faire l'équipe lorsqu'un critère de sortie d'une phase SDLC n'est pas satisfait ?**
   a. Le cacher pour ne pas ralentir.
   b. Passer quand même à l'implémentation.
   c. Demander au support de décider seul.
   d. Le rendre visible, documenter l'écart et revenir à la bonne phase si nécessaire.

Étape 2 — Spécifications comme langage commun

> **Idée centrale :** une spécification décrit précisément ce qui doit être vrai sous condition. Elle peut exister dans un document, un modèle, un diagramme, un test Xray, un contrat d'interface, une configuration, du code, une preuve automatisée ou un signal observable. La forme cible est **IF... THEN...**.

Compétences attendues

Compétence	Ce que le participant doit savoir faire	Référence principale
Découvrir une spécification	Identifier les comportements attendus cachés dans un besoin, un incident, une demande client, une règle métier, une contrainte, une configuration ou un signal terrain.	SPÉCIFICATION
Comprendre une spécification	Distinguer condition, contexte, entrée, état, événement, dépendance, résultat observable, contrainte et preuve attendue.	SPÉCIFICATION
Écrire en IF... THEN...	Formuler une spécification claire, atomique, observable, vérifiable et exploitable par produit, développement, validation, support, IA et automatisation.	SPÉCIFICATION

Relier spécification et preuve	Déduire quelles validations, signaux, métriques, logs, traces ou observations peuvent prouver que le comportement attendu est vrai.	Exemples concrets de combinaisons

Questions de maîtrise

Thème	Question à savoir répondre	Ce que la réponse doit démontrer
Définition	Qu'est-ce qu'une spécification ?	Une spécification décrit précisément ce qui doit être vrai, sous condition, avec un comportement observable et vérifiable.
Différence avec une intention	En quoi une spécification diffère-t-elle d'un souhait, d'une story ou d'une exigence vague ?	Une intention exprime pourquoi ou ce qu'on aimerait; une spécification rend explicite le comportement attendu et sa preuve.
Atomicité	La spécification décrit-elle un seul comportement vérifiable ou plusieurs comportements mélangés ?	Le participant sait découper une spécification trop large en éléments plus petits et testables.
Condition	Qu'est-ce qui doit être vrai avant que le comportement attendu s'applique : état, configuration, profil, environnement, dépendance, événement ?	Le participant sait clarifier le « IF » plutôt que d'écrire seulement le résultat souhaité.
Observable	Comment saura-t-on que le comportement attendu est réellement arrivé ?	Le participant sait définir un « THEN » visible, mesurable ou traçable.

Cas limites	Quels seuils, valeurs extrêmes, états inhabituels, erreurs, dépendances lentes ou absentes doivent être couverts ?	Le participant sait compléter le comportement nominal par les limites et exceptions importantes.
Traçabilité	À quel besoin, signal, incident, risque, décision, test, modèle ou preuve cette spécification doit-elle être reliée ?	Le participant comprend qu'une spécification doit rester connectée au flux SDLC.

Quiz bloc 2 — Spécifications

1. **Quelle définition décrit le mieux une spécification exploitable dans le SDLC ?**
   a. Une idée générale de ce que le client aimerait avoir.
   b. Une tâche technique assignée à un développeur.
   c. Une condition et un comportement attendu observable, formulés de façon vérifiable.
   d. Une note de réunion non liée à une preuve.
2. **Quelle formulation est la meilleure spécification IF… THEN… parmi les choix suivants ?**
   a. Le système doit être rapide.
   b. L'utilisateur veut une meilleure expérience.
   c. Améliorer la validation du modèle.
   d. IF le modèle est invalide au lancement, THEN le lancement est bloqué et un message explicite indique la cause.
3. **Que faire avec une spécification vague comme « gérer les erreurs correctement » ?**
   a. La transformer en conditions, erreurs observables, comportements attendus, messages, états et preuves.
   b. L'accepter telle quelle, car elle exprime une intention qualité.
   c. La supprimer parce qu'elle parle d'erreurs.
   d. La déplacer directement en DIFFUSER.
4. **Quelle question aide le mieux à découvrir des spécifications manquantes ?**
   a. Qui peut coder cela le plus vite ?
   b. Peut-on éviter d'en parler maintenant ?
   c. Quels comportements doivent être vrais dans les cas nominaux, limites, erreurs, dépendances et environnements supportés ?
   d. Quelle couleur devrait avoir le bouton ?

5. **Quel indice montre qu'une spécification est assez mûre pour alimenter un atelier DÉFINIR ?**
   a. Elle est compréhensible seulement par l'expert du domaine.
   b. Elle contient une solution technique sans contexte.
   c. Elle évite toute discussion avec IV, support ou architecture.
   d. Elle permet de discuter des preuves attendues, des niveaux de validation, des risques et des cas limites.

Étape 3 — Catégorisation et stratégie de validation

Une spécification de qualité ne dit pas seulement quoi construire. Elle aide aussi à déterminer comment prouver que le comportement est correct. Pour cela, elle doit pouvoir être catégorisée selon quatre axes : **niveau d'intégration**, **interface observée**, **angle d'observation** et **type de validation**.

Axe	Question clé	Exemples
Niveau d'intégration	Jusqu'où assemble-t-on réellement pour observer le comportement ?	Small, medium, large.
Interface observée	Par quelle frontière observe-t-on le système ?	Interface utilisateur, interface applicative, interface machine.
Angle d'observation	Quelle visibilité interne utilise-t-on pour comprendre ou prouver le comportement ?	Black-box, gray-box, white-box.
Type de validation	Quelle propriété ou quel risque cherche-t-on à prouver ?	Fonctionnel, contrat, intégrité, performance, sécurité, résilience, compatibilité, santé, UI, UX, utilisabilité.

Quiz bloc 3 — Catégorisation

1. **Pourquoi catégoriser une spécification selon les quatre axes de validation ?**
   a. Pour rendre la documentation plus longue.
   b. Pour nommer précisément ce qui est validé, à quelle frontière, avec quelle visibilité et pour quelle propriété.
   c. Pour remplacer toutes les discussions produit.

d. Pour décider automatiquement qui sera assigné au ticket.

2. **Parmi les choix suivants, lequel est un niveau d'intégration ?**
   a. Black-box.
   b. Performance.
   c. Medium.
   d. Interface machine.
3. **Parmi les choix suivants, lequel est une interface observée ?**
   a. Small.
   b. Interface applicative.
   c. Gray-box.
   d. Résilience.
4. **Parmi les choix suivants, lequel est un angle d'observation ?**
   a. White-box.
   b. Large.
   c. Contrat.
   d. Sécurité.
5. **Parmi les choix suivants, lequel est un type de validation ?**
   a. Small.
   b. Interface utilisateur.
   c. Black-box.
   d. Intégrité des données.

Étape 4 — Modélisation produit, mockups, designs et MBT

Principe : le modèle est une représentation visuelle ou comportementale des spécifications; la spécification est l'expression textuelle du comportement attendu. Les deux doivent raconter la même histoire. Une divergence entre mockup, design, diagramme, modèle comportemental, couverture testable et spécification IF... THEN... est un défaut de définition à corriger avant PLANIFIER. Les outils cités dans cette section sont des exemples; le besoin principal est d'obtenir une chaîne standardisée et fiable entre produit, livraison, validation, automatisation et apprentissage.

Attention — les technologies citées sont des exemples : yEd, GraphWalker, GraphWalker Studio, AltWalker, Jira, Xray, Robot Framework, pytest, Eggplant ou d'autres solutions comparables ne sont pas l'objectif en soi. Le besoin à combler est plus important que l'outil : uniformiser et standardiser la livraison, documenter le produit par des spécifications

fiables, relier ces spécifications aux modèles, aux preuves, à l'automatisation et à l'IA, puis maintenir cette chaîne comme une documentation vivante. Contrairement à une approche de maturité séquentielle « à l'ancienne », il n'est pas nécessaire d'attendre d'avoir atteint un niveau élevé de maturité avant de commencer. L'objectif est d'établir une fondation minimale viable qui permet de progresser, d'apprendre et d'automatiser graduellement.

Pour documenter sérieusement le produit, il faut combiner plusieurs capacités : une documentation produit testable, une modélisation comportementale compréhensible, une traçabilité vers la livraison, une automatisation progressive et des sources fiables pour l'IA. Xray peut être une bonne base côté documentation produit testable; GraphWalker, AltWalker, yEd/yEd Live, GraphWalker Studio, Eggplant ou d'autres solutions MBT peuvent soutenir la modélisation comportementale. L'objectif n'est toutefois pas d'imposer une technologie unique : l'objectif est de faire converger documentation vivante, spécifications exécutables, modèles visuels, preuves de validation, automatisation et apprentissage continu.

Questions / réponses théoriques

Question	Réponse attendue	Pourquoi c'est important avant les ateliers
Pourquoi modéliser ou faire un diagramme avant de planifier ?	Parce qu'un modèle rend visibles les flux, états, dépendances, transitions, exceptions et interactions qui restent souvent implicites dans le texte.	Les participants apprennent à détecter les trous de compréhension avant qu'ils deviennent des défauts de code, de test ou d'intégration.
Un mockup ou un design UI est-il une spécification ?	Oui, s'il décrit un comportement attendu, une règle d'affichage, une interaction, un état ou une contrainte observable. Il doit toutefois être relié aux spécifications IF… THEN… qui explicitent les conditions et résultats attendus.	Le participant évite de traiter le design comme une simple image décorative ou comme une vérité non testable.
Pourquoi lier les spécifications aux modèles, diagrammes ou designs ?	Pour assurer la traçabilité entre besoin, comportement attendu, représentation	Sans lien, chacun peut interpréter différemment le

	visuelle, implémentation et preuve de validation.	scope, les interactions ou les cas limites.
Que faire si un diagramme et une spécification ne concordent pas ?	Ne pas choisir arbitrairement l'un des deux. Il faut revenir à DÉFINIR, clarifier la divergence, mettre à jour les artefacts et valider la concordance.	Une divergence modèle ↔ texte est un signal que l'équipe ne partage pas encore la même compréhension.
Quels modèles sont utiles selon le type de comportement ?	Un flux aide à représenter un parcours; une machine à états aide à représenter les transitions; un diagramme de séquence aide à représenter les interactions; un modèle de domaine aide à clarifier les concepts; un mockup aide à clarifier l'expérience et les états UI.	Le participant apprend à choisir la représentation qui réduit le mieux l'ambiguïté.
Comment relier un modèle à une spécification IF… THEN… ?	Chaque branche, état, transition, interaction, erreur ou règle visible dans le modèle devrait correspondre à une ou plusieurs spécifications observables.	Cette pratique permet de vérifier la couverture : rien d'important dans le modèle ne devrait rester non spécifié.
Pourquoi parler de MBT et de modèles comportementaux, peu importe l'outil choisi ?	Parce que le besoin est de représenter les comportements produit de façon vivante, lisible, traçable et exploitable. GraphWalker, AltWalker, Eggplant ou d'autres outils MBT sont des exemples de moyens possibles pour relier parcours, règles, états, transitions, spécifications et preuves.	Le participant comprend que la cible n'est pas l'adoption d'un outil précis, mais la capacité à maintenir une documentation produit qui soutient la validation, l'automatisation, la couverture et les décisions.

Comment choisir l'écosystème d'outils approprié ?	Le choix doit partir du besoin : édition visuelle accessible aux PO/QA, intégration développeur et CI/CD, exécution automatisée, couverture, maintenabilité, modularité et capacité à relier le modèle aux spécifications. yEd/yEd Live, GraphWalker Studio, AltWalker, Eggplant ou d'autres solutions peuvent être pertinentes selon le contexte.	Les participants apprennent à évaluer une solution selon la chaîne de valeur produit plutôt que selon une préférence d'outil.
Pourquoi yEd ou yEd Live restent-ils importants historiquement ?	Ils permettent de créer facilement des graphes GraphML compatibles GraphWalker avec une interface visuelle simple. Cela rend la modélisation comportementale accessible au métier, aux QA et aux facilitateurs d'atelier sans imposer immédiatement un environnement de développement.	Un atelier peut démarrer la modélisation à partir d'un support visuel compréhensible, puis relier progressivement le modèle aux spécifications et à l'automatisation.
Qu'apporte AltWalker par rapport à une utilisation GraphWalker plus classique ?	AltWalker représente une évolution intéressante parce qu'il est plus orienté développeur : support Python, meilleure intégration CI/CD, scripts et outils dédiés comme le Model Editor et l'extension VS Code pour visualiser, manipuler et maintenir les modèles dans l'environnement de développement.	Les développeurs peuvent intégrer le modèle au workflow d'ingénierie, aux pipelines et au code de test sans couper le lien avec la documentation produit.

Comment GraphWalker + AltWalker répondent-ils aux besoins des différents rôles ?	Les PO et QA bénéficient d'une modélisation visuelle compréhensible; les développeurs travaillent avec des outils intégrés à l'IDE, aux scripts et à l'automatisation; les architectes structurent une solution modulaire, réutilisable et scalable autour du MBT.	Le même modèle peut devenir un point de convergence entre compréhension produit, stratégie de validation, architecture de test et automatisation.
Qu'est-ce que le Model-Based Testing (MBT) apporte au SDLC ?	Le MBT permet de générer dynamiquement des scénarios de test à partir des modèles comportementaux. Les chemins du modèle deviennent des parcours de validation, ce qui améliore la couverture et réduit les scénarios écrits manuellement en double.	Les participants comprennent qu'un modèle bien structuré peut devenir une source de scénarios, de preuves et de discussion de couverture.
Comment GraphWalker se relie-t-il à l'automatisation ?	Les nœuds et transitions du modèle peuvent être liés à des étapes d'automatisation dans Robot Framework, pytest ou d'autres frameworks. Le modèle orchestre le parcours; le code d'automatisation exécute les actions et vérifications concrètes.	Cette séparation aide à garder le modèle compréhensible par les parties prenantes tout en permettant une exécution technique robuste.

Quel est le lien entre GraphWalker, AltWalker, Xray et Jira ?	Jira pilote la livraison, Xray centralise les spécifications testables et la couverture, GraphWalker ou AltWalker portent les modèles comportementaux exécutables. Les quatre doivent être reliés, mais ne doivent pas se dupliquer.	Cette séparation évite de transformer Jira en documentation produit complète, évite que Xray devienne une archive isolée et évite que les modèles deviennent des dessins sans preuve.
Pourquoi l'investissement initial en modélisation / MBT est-il acceptable ?	Parce qu'une fondation minimale viable — quelques parcours critiques modélisés, reliés aux spécifications et aux preuves — permet déjà de réduire les ambiguïtés, d'améliorer la couverture et de préparer l'automatisation et l'IA. Il ne s'agit pas d'attendre une maturité parfaite avant de commencer, mais de commencer petit, avec une base standardisée et améliorable.	Les participants comprennent que l'objectif n'est pas de tout modéliser immédiatement, mais d'installer une capacité durable : spécifications fiables, modèles reliés, preuves traçables, automatisation progressive et apprentissage continu.

Quiz bloc 4 — Modélisation produit et MBT

1. **Dans cette formation, qu'est-ce qu'un modèle produit doit représenter principalement ?**
   a. Un élément décoratif à ajouter si le temps le permet.
   b. Un remplacement complet des spécifications IF... THEN....
   c. Une représentation visuelle des comportements, flux, états ou dépendances à aligner avec les spécifications.
   d. Un artefact réservé uniquement à l'architecture.
2. **Un mockup montre un bouton désactivé dans un certain état. Quel est le bon réflexe de spécification ?**
   a. Considérer que l'image suffit toujours.
   b. Ignorer le mockup, car seul le code compte.

c. Créer ou lier une spécification qui précise IF quel état est vrai, THEN le bouton est désactivé et l'utilisateur comprend pourquoi.

d. Attendre la diffusion pour valider.

3. **Que signifie la concordance entre modèle, diagramme, design et spécifications ?**
   a. Le modèle peut raconter une histoire et le texte une autre.
   b. Le diagramme remplace les preuves de validation.
   c. La concordance n'est nécessaire qu'après la release.
   d. Le modèle, le diagramme, le design et les spécifications décrivent le même comportement attendu.
4. **Un modèle montre une dépendance externe indisponible. Quelle discussion doit en découler ?**
   a. Une couleur différente dans le diagramme.
   b. Une discussion sur les cas limites, la résilience et le comportement attendu en cas de dépendance indisponible.
   c. Un sprint plus long sans autre clarification.
   d. Une décision Go / No-Go immédiate.
5. **Quel est l'intérêt d'un outil de modélisation comportementale comme GraphWalker ou un équivalent MBT ?**
   a. Remplacer Jira comme outil de livraison.
   b. Remplacer toutes les validations Xray par des dessins non exécutables.
   c. Servir de solution centrale de modélisation orientée comportement, documentation vivante et base de spécifications exécutables.
   d. Créer automatiquement les EPIC et FEATURE sans gouvernance.
6. **Quel est l'apport principal du Model-Based Testing (MBT) ?**
   a. Éviter complètement d'écrire des spécifications.
   b. Supprimer les besoins de validation humaine.
   c. Remplacer le SDLC officiel.
   d. Générer dynamiquement des scénarios de test à partir des chemins du modèle.
7. **Quelle séparation saine faut-il conserver entre Jira, Xray et les modèles comportementaux ?**
   a. Les trois outils doivent contenir les mêmes informations copiées-collées.
   b. GraphWalker remplace les stories Jira.
   c. Jira pilote la livraison, Xray porte les spécifications testables et la couverture, GraphWalker porte les modèles comportementaux exécutables.

d. Xray devient inutile dès qu'un modèle existe.

8. **Quel type de besoin un outil comme AltWalker peut-il mieux couvrir dans l'écosystème MBT ?**
   a. AltWalker remplace les besoins de modélisation visuelle et de validation produit.
   b. AltWalker sert uniquement à dessiner des mockups UI.
   c. AltWalker remplace Jira comme système de livraison.
   d. AltWalker apporte une approche plus developer-friendly avec Python, CI/CD, Model Editor et extension VS Code pour visualiser et manipuler les modèles.
9. **Pourquoi des outils visuels comme yEd ou yEd Live peuvent-ils être utiles même dans une trajectoire MBT ?**
   a. Pour éviter de relier les modèles aux spécifications.
   b. Pour permettre aux profils PO, QA et métier de contribuer à des modèles visuels compréhensibles, tout en gardant une trajectoire vers l'exécution MBT.
   c. Pour remplacer tous les tests Xray par des images.
   d. Pour empêcher les développeurs d'automatiser les parcours.
10. **Quel est le bénéfice principal d'un écosystème de modèles reliés aux spécifications, à l'automatisation et à la couverture ?**
    a. Le même modèle peut unir compréhension produit, documentation vivante, architecture de test, automatisation et couverture évolutive.
    b. Chaque rôle utilise un artefact différent sans lien avec les autres.
    c. Les modèles deviennent inutiles une fois les stories créées.
    d. L'architecture n'a plus besoin de modularité.

Étape 5 — Xray, test sets et couverture comme documentation produit testable

Dans une formation théorique, Xray doit être compris comme un mécanisme de **documentation produit testable** lorsqu'il est utilisé avec rigueur. Un test case Xray formulé comme une spécification observable peut devenir une source de vérité testable; les test sets permettent de regrouper ces spécifications par périmètre; le test coverage permet de relier un Jira au scope des comportements à livrer et à prouver.

Questions / réponses théoriques

Question	Réponse attendue	Pourquoi c'est important avant les ateliers
Est-ce qu'un test case Xray peut être une spécification ?	Oui, s'il décrit précisément un comportement attendu, sous	Le participant comprend que le mot « test » ne limite pas

	condition, avec un résultat observable et vérifiable.	l'artefact à l'exécution; il peut aussi porter la définition produit testable.
À quoi servent les test sets Xray ?	À regrouper des spécifications testables par périmètre utile : feature, component, page, domaine, capacité, risque ou parcours.	Ils aident à naviguer la documentation produit testable sans dupliquer l'information dans plusieurs champs Jira.
Comment le test coverage aide-t-il à scope-er un Jira ?	Il montre quelles spécifications et validations sont liées au périmètre de l'epic, feature, story ou bug.	Le scope devient observable par la couverture, plutôt que caché dans une longue description ou des champs personnalisés.
Quelle est la séparation saine entre Jira et Xray ?	Jira pilote la livraison; Xray formalise les spécifications testables, validations et preuves de couverture.	Cette séparation évite que Jira devienne un référentiel produit lourd, fragmenté et difficile à maintenir.
Quel risque demeure si Xray est mal utilisé ?	Xray peut devenir une archive de tests déconnectée si les test cases ne sont pas formulés comme spécifications, reliés aux Jira et maintenus avec le produit.	L'objectif n'est pas d'avoir plus de tests, mais une définition produit testable, traçable et utile aux décisions.

Quiz bloc 5 — Xray, test sets et couverture

1. **Quel rôle Xray peut-il jouer lorsqu'il est utilisé comme documentation produit testable ?**
   a. Xray sert seulement à exécuter des tests après le développement.
   b. Xray peut servir de référentiel de spécifications testables lorsque les tests, test sets et validations décrivent les comportements attendus.
   c. Xray remplace complètement Jira, Confluence et les ateliers.
   d. Xray sert principalement à assigner les développeurs.
2. **Pourquoi utiliser des test sets par feature, component, page ou domaine ?**
   a. Ils permettent de centraliser et regrouper les spécifications testables par périmètre produit compréhensible.

b. Ils évitent d'écrire des spécifications IF… THEN….
c. Ils servent à contourner la gouvernance Jira.
d. Ils remplacent la stratégie de validation.

3. **Comment le test coverage aide-t-il à scope-er un Jira ?**
   a. Il remplit automatiquement tous les champs Jira.
   b. Il montre quelles spécifications et validations couvrent le périmètre de livraison concerné.
   c. Il remplace la Definition of Ready.
   d. Il ferme automatiquement les tickets lorsque les tests existent.
4. **Pourquoi éviter de mettre toute la définition produit dans des champs Jira ?**
   a. Parce que Jira ne devrait jamais contenir de lien.
   b. Parce que Jira doit surtout piloter la livraison, alors que la définition produit testable devrait être reliée aux spécifications et validations.
   c. Parce que les développeurs ne lisent jamais Jira.
   d. Parce que Xray doit contenir seulement des résultats d'exécution.
5. **Quel est l'anti-pattern principal à éviter avec les test sets ?**
   a. Les regrouper par feature, component, page ou domaine.
   b. Les relier aux Jira pertinents.
   c. Les utiliser comme inventaire figé et non maintenu, déconnecté des spécifications, des stories et des résultats.
   d. Les consulter avant un atelier DÉFINIR.

Étape 6 — Gouvernance Jira, découpage des livraisons et tableaux

Avant les ateliers, les participants doivent comprendre que Jira n'est pas seulement une liste de tâches : c'est le système de pilotage de la livraison. Sans règles communes de hiérarchie, de versionnement, de découpage et de tableaux, les équipes peuvent rapidement créer une explosion de tickets, de vues, de statuts, de champs et de projets incohérents — un effet « Ragnarok » où la traçabilité devient presque impossible à maintenir.

> **Principe central :** rester dans un même espace/projet Jira produit avec une hiérarchie simple — **EPIC** comme grand-parent, **FEATURE** comme parent et **STORY / BUG** comme enfant — permet de préserver la traçabilité entre valeur affaire, valeur produit, réalisation, validation, version et preuve qualité.

Questions / réponses théoriques

Question	Réponse attendue	Pourquoi c'est important avant les ateliers
Pourquoi utiliser un même projet ou espace Jira pour le produit ?	Parce qu'un même projet Jira produit permet de conserver une vue cohérente du flux, des versions, des dépendances, des liens Xray, de la couverture, des tableaux et des métriques.	Si chaque équipe ou périmètre crée son propre projet, la traçabilité se fragmente et les ateliers passent du temps à réconcilier les outils au lieu de clarifier le produit.
Pourquoi garder la hiérarchie EPIC → FEATURE → STORY / BUG ?	Parce que cette hiérarchie relie la valeur affaire, la valeur produit et la réalisation concrète. Une STORY seule, une FEATURE seule ou un BUG orphelin perd son contexte.	Les participants doivent comprendre qu'un ticket sans parent est difficile à prioriser, scope-er, valider, expliquer et relier à une version.
Pourquoi éviter de multiplier les types Jira ?	Parce que trop de types — request, requirement, requis, demande, initiative locale, etc. — créent des taxonomies parallèles et rendent les requêtes, tableaux, métriques et responsabilités incohérents.	En atelier, tout le monde doit parler le même langage. La complexité de l'outil ne doit pas remplacer la clarté du découpage produit.
Pourquoi recommander une seule FEATURE par release ou version mineure ?	Parce que la FEATURE représente l'unité principale de livraison produit. Une relation proche de 1:1 entre FEATURE et version mineure réduit la complexité, facilite la validation et rend la release explicable.	Les participants doivent comprendre qu'une release qui mélange plusieurs features non reliées augmente fortement les risques, la charge de validation et la difficulté de support.

Pourquoi privilégier de petites releases avant tout ?	Parce qu'un petit paquet de livraison est plus facile à comprendre, tester, diffuser, diagnostiquer, corriger et confirmer. Le risque augmente rapidement lorsque plusieurs changements sont regroupés.	Le réflexe recherché est de réduire le batch size plutôt que de repousser les validations dans une grosse livraison difficile à maîtriser.
Pourquoi un tableau SDLC produit est-il nécessaire ?	Parce qu'il donne une vue du flux produit complet : À FAIRE, ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER, CONFIRMER et TERMINÉ.	Les ateliers doivent pouvoir situer un item dans sa maturité réelle, pas seulement dans son état d'exécution d'équipe.
Pourquoi garder des tableaux d'équipes dans le même projet Jira ?	Parce que les équipes peuvent avoir des vues opérationnelles différentes tout en partageant le même référentiel, les mêmes tickets, la même hiérarchie et les mêmes liens de traçabilité.	On évite de dupliquer les items dans plusieurs projets tout en permettant aux équipes de gérer leur quotidien selon leur besoin.
Quel est le risque si on ne fait pas cela ?	On obtient une explosion de projets, types, statuts, champs, tableaux, versions et tickets orphelins. Le système devient difficile à gouverner, comparer, auditer et améliorer.	Les participants doivent reconnaître que cette explosion n'est pas seulement un problème d'outil : elle détruit la visibilité du flux SDLC et la confiance dans les métriques.
Quel lien faire avec Xray et les spécifications ?	Jira porte la livraison et Xray porte la définition produit testable. Les tickets Jira doivent être reliés aux spécifications, test cases, test sets et couvertures Xray plutôt que contenir toute la définition produit dans des champs.	Cette séparation réduit la duplication et permet aux ateliers de discuter du scope à partir de la couverture et des preuves attendues.

Quiz bloc 6 — Gouvernance Jira et découpage de livraison

1. **Pourquoi conserver un même projet Jira produit plutôt que multiplier les projets par équipe ou périmètre ?**
   a. Pour empêcher les équipes d'avoir leurs propres tableaux.
   b. Pour remplacer Xray et Confluence.
   c. Pour conserver une traçabilité commune entre hiérarchie, versions, flux, dépendances, couverture et métriques.
   d. Pour éviter toute discussion de priorisation.
2. **À quoi sert la hiérarchie EPIC → FEATURE → STORY / BUG ?**
   a. Créer plus de tickets pour suivre le travail.
   b. Permettre aux stories d'exister sans contexte.
   c. Relier valeur affaire, valeur produit et réalisation concrète.
   d. Remplacer les spécifications IF… THEN….
3. **Pourquoi les tickets orphelins sont-ils problématiques ?**
   a. Parce qu'ils sont toujours techniquement invalides dans Jira.
   b. Parce qu'ils empêchent automatiquement les builds.
   c. Parce qu'ils perdent leur contexte produit, leur priorité, leur scope et leur traçabilité de validation.
   d. Parce qu'ils doivent toujours être convertis en tâches personnelles.
4. **Pourquoi viser une relation proche de 1 FEATURE pour 1 release ou version mineure ?**
   a. Pour réduire la complexité, faciliter la validation et rendre la valeur livrée plus explicable.
   b. Pour rendre la release plus grosse et plus visible.
   c. Pour éviter d'utiliser des stories.
   d. Pour empêcher les correctifs patch.
5. **Pourquoi privilégier les petites releases ?**
   a. Elles éliminent tout besoin de validation.
   b. Elles permettent d'ignorer les dépendances.
   c. Elles réduisent le risque, facilitent le diagnostic et accélèrent la confirmation terrain.
   d. Elles remplacent le besoin de critères de sortie.
6. **Quelle est la bonne relation entre tableau SDLC produit et tableaux d'équipes ?**
   a. Le tableau produit montre le flux SDLC global; les tableaux d'équipes offrent des vues opérationnelles sur les mêmes tickets dans le même projet.
   b. Chaque équipe doit avoir son propre projet séparé et ses propres copies de tickets.

c. Le tableau produit remplace tous les tableaux d'équipes.
d. Les tableaux d'équipes doivent utiliser des statuts incompatibles avec le SDLC.

7. **Qu'est-ce que l'effet « Ragnarok » dans la gouvernance Jira ?**
   a. Un seul projet produit, une hiérarchie claire et des tableaux filtrés.
   b. Une FEATURE bien reliée à une version mineure.
   c. Une explosion de projets, types, statuts, champs, versions, tableaux et tickets orphelins.
   d. Une story liée à ses spécifications Xray.
8. **Quelle séparation saine faut-il conserver entre Jira et Xray ?**
   a. Jira définit le produit en détail et Xray sert seulement à archiver les résultats.
   b. Xray remplace toutes les stories Jira.
   c. Jira pilote la livraison; Xray formalise les spécifications testables, la couverture et les preuves.
   d. Jira et Xray doivent contenir les mêmes informations copiées-collées.

Étape 7 — Qualimétrie, automatisation et IA

Cette étape comble le lien entre théorie qualité et passage à l'échelle. Les participants doivent comprendre que la qualimétrie, l'automatisation et l'IA ne remplacent pas la clarification : elles amplifient ce qui est déjà structuré. Si les spécifications, modèles et sources de vérité sont flous, l'automatisation et l'IA amplifient le flou.

Questions / réponses théoriques

Question	Réponse attendue	Pourquoi c'est important avant les ateliers
Pourquoi mesurer la qualité ?	Pour observer si le système de livraison réduit réellement les reprises, ambiguïtés, risques résiduels, défauts échappés, délais de confirmation et écarts de couverture.	Les ateliers doivent produire des décisions et artefacts mesurables, pas seulement des impressions.
Que faut-il mesurer en priorité ?	Les signaux liés au flux et à la preuve : couverture des spécifications, critères de sortie satisfaits, défauts liés à l'ambiguïté, délais de validation, taux de reprise,	Le participant comprend que la métrique doit servir la décision, pas créer un tableau de bord décoratif.

	risques résiduels et signaux terrain CARE.	
Pourquoi l'automatisation dépend-elle de bonnes spécifications ?	Parce qu'un test automatisé fiable doit prouver un comportement attendu clair. Automatiser une ambiguïté crée seulement une ambiguïté qui s'exécute plus vite.	Les participants comprennent pourquoi DÉFINIR et la modélisation précèdent l'automatisation sérieuse.
Quand automatiser ?	Lorsque le comportement est assez stable, observable, répétable, à risque ou coûteux à valider manuellement, et que la preuve attendue est claire.	On évite d'automatiser trop tôt des hypothèses instables ou trop tard des validations critiques.
Quel lien entre CI/CD et Continuous Specification ?	Le CI/CD exécute et diffuse; la Continuous Specification clarifie ce qui doit être vrai et prouvé. Sans spécifications fiables, le pipeline accélère seulement le passage du flou vers la production.	Les participants comprennent que la qualité ne commence pas au pipeline : elle commence à la définition du comportement attendu.
Comment utiliser l'IA de façon saine ?	L'IA peut aider à analyser du code, résumer un ticket, générer des hypothèses, proposer des cas de validation ou repérer des incohérences, mais les sources de vérité et décisions doivent rester validées humainement.	Les ateliers doivent produire des entrées fiables pour l'IA : spécifications, modèles, décisions, liens et preuves.

Quiz bloc 7 — Qualimétrie, automatisation et IA

1. **Pourquoi mesurer la qualité et le flux SDLC ?**
   a. Pour produire des chiffres sans lien avec les décisions.
   b. Pour remplacer les ateliers.
   c. Pour observer si les pratiques réduisent les reprises, ambiguïtés, défauts échappés et risques résiduels.

d. Pour décider automatiquement de toutes les priorités.

2. **Qu'est-ce qui rend une métrique qualité utile ?**
   a. Elle aide à comprendre un risque, une décision, un apprentissage ou une amélioration du flux.
   b. Elle est facile à produire, même si personne ne s'en sert.
   c. Elle remplace les preuves de validation.
   d. Elle doit toujours être maximisée, peu importe le contexte.
3. **Pourquoi est-il risqué d'automatiser une spécification floue ?**
   a. Parce que l'automatisation ne sert jamais à la qualité.
   b. Parce que seuls les tests manuels sont acceptables.
   c. Parce qu'on obtient une ambiguïté qui s'exécute plus vite et donne une fausse impression de maîtrise.
   d. Parce que l'automatisation doit attendre CARE.
4. **Quand l'automatisation devient-elle pertinente ?**
   a. Dès qu'une idée est évoquée.
   b. Quand il n'existe aucune preuve attendue.
   c. Uniquement après plusieurs releases échouées.
   d. Quand le comportement est clair, répétable, observable, à risque ou coûteux à valider manuellement.
5. **Quel est le risque d'utiliser l'IA sur des sources floues ou non validées ?**
   a. L'IA refusera toujours de répondre.
   b. L'IA remplace automatiquement le RACI.
   c. L'IA peut amplifier ou reformuler proprement des ambiguïtés, incohérences et hypothèses non validées.
   d. L'IA rend les spécifications inutiles.

Étape 8 — Intégration dans les phases SDLC

Cette étape relie les acquis théoriques aux phases SDLC sans remplacer les pages officielles. L'objectif est de comprendre ce que les spécifications, modèles, preuves, couvertures et signaux apportent à chaque phase.

Phase	Contribution théorique attendue	Références principales
ÉLICITER	Découvrir les signaux, besoins, symptômes, hypothèses,	Comment éliciter

	impacts et questions qui deviendront des spécifications à clarifier.	
DÉFINIR	Transformer la compréhension en spécifications atomiques, modélisées, testables, traçables et approuvées.	Comment définir SPÉCIFICATION
PLANIFIER	Vérifier que le scope, les dépendances, risques, preuves attendues et critères de readiness sont assez clairs pour engager le travail.	Comment planifier
IMPLÉMENTER	Préserver la traçabilité entre spécifications, modèles, code, tests, revues, automatisation et Definition of Done.	Comment implémenter
DIFFUSER	S'appuyer sur les preuves, risques résiduels, supportabilité, observabilité et rollback pour décider.	Comment diffuser Contrat qualité
CONFIRMER	Comparer les signaux terrain aux comportements spécifiés et alimenter la boucle d'amélioration continue.	Comment confirmer Qualimétrie

Examen sommaire

Contexte : une fonctionnalité est proposée pour un prochain sprint. La valeur client est confirmée, mais les spécifications IF… THEN… ne couvrent pas les cas limites, le mockup UI montre des états non décrits dans les spécifications, l'IV n'a pas confirmé la stratégie de validation et CARE n'a pas été consulté sur les impacts client.

1. **Dans le contexte présenté, quel est le diagnostic SDLC le plus juste ?**
   a. Le sujet est prêt pour IMPLÉMENTER.
   b. Le sujet devrait être diffusé rapidement.
   c. Le sujet tente de passer vers PLANIFIER alors que DÉFINIR est incomplet.

d. Le sujet ne concerne pas le SDLC.

2. **Quels éléments manquent le plus clairement avant de pouvoir planifier correctement ?**
   a. La valeur client.
   b. La présence d'un PO.
   c. La stratégie de validation, les cas limites et la concordance mockup ↔ spécifications.
   d. Le fait qu'un sprint existe.
3. **Quel est le bon réflexe de l'équipe avant d'engager l'implémentation ?**
   a. Retourner ou rester à DÉFINIR avec actions explicites.
   b. Engager l'implémentation et documenter après.
   c. Annuler automatiquement la fonctionnalité.
   d. Diffuser une version partielle sans validation.
4. **Quelles références théoriques sont les plus utiles pour combler les écarts identifiés ?**
   a. Seulement Qualimétrie.
   b. SPÉCIFICATION, Comment définir et Les tests — niveau × interface × angle × type.
   c. Seulement la page de release notes.
   d. Aucune référence n'est nécessaire.
5. **Quel anti-pattern est illustré par ce cas ?**
   a. Rendre les risques visibles trop tôt.
   b. Consulter trop de références officielles.
   c. Planifier alors que les critères de définition ne sont pas satisfaits.
   d. Documenter les décisions au bon endroit.

Étape 9 — Examen récapitulatif

Contexte : un incident client révèle un comportement problématique. Le signal initial est clair, mais la cause racine est incertaine. Deux options techniques sont proposées, une dépendance inter-produit est possible, un diagramme de séquence montre une interaction non documentée, un modèle GraphWalker n'existe pas encore pour ce parcours critique, et une date de release est déjà évoquée.

1. **Dans ce cas d'incident client, quel élément doit être clarifié en premier en ÉLICITER ?**
   a. La date de release.
   b. La branche de développement.
   c. Le signal, la source, l'impact, les faits et les hypothèses RCA.
   d. Le message de release note.
2. **Pourquoi deux options techniques ne suffisent-elles pas pour engager la solution ?**

a. Parce que les développeurs ne doivent jamais proposer de solution.
b. Parce que le PO doit toujours choisir seul.
c. Parce que le comportement attendu, les impacts, les risques, les modèles et les preuves ne sont pas encore suffisamment définis.
d. Parce que toute option technique doit attendre la diffusion.

3. **Que devrait produire la phase DÉFINIR avant de passer à PLANIFIER ?**
   a. Une décision Go / No-Go finale.
   b. Des spécifications observables, modèles concordants, cas limites, impacts, risques et preuves attendues.
   c. Un rapport post-livraison.
   d. Une liste de participants seulement.
4. **Quand devient-il pertinent de passer à PLANIFIER ?**
   a. Quand les critères de sortie de DÉFINIR sont suffisamment satisfaits et que la DoR peut être évaluée.
   b. Dès qu'une date est demandée.
   c. Dès qu'une option technique semble intéressante.
   d. Quand le support cesse de poser des questions.
5. **Quelle trace utile devrait rester après l'atelier théorique ou la discussion de cadrage ?**
   a. « On verra plus tard. »
   b. Une liste informelle de commentaires non assignés.
   c. Décision, justification, risques, actions, owners, références, spécifications impactées, modèles liés et prochaine étape.
   d. Un résumé sans décision.

Anti-patterns à reconnaître

Thème	Anti-pattern	Bon réflexe
SDLC	Voir le SDLC comme une procédure administrative.	Le voir comme une boucle de réduction de risque et d'apprentissage.
ÉLICITER	Sauter à la solution avant de qualifier le signal.	Distinguer faits, hypothèses, impacts, source et décision de poursuite.

DÉFINIR	Écrire des spécifications vagues ou non observables.	Formuler en IF… THEN…, clarifier conditions, résultats, preuves et cas limites.
Modèles	Laisser diagrammes, mockups, designs, GraphWalker et spécifications diverger.	Valider la concordance modèle ↔ texte ↔ preuve avant PLANIFIER.
MBT	Traiter GraphWalker comme un dessin isolé ou comme un outil technique réservé à l'automatisation.	L'utiliser comme modèle comportemental vivant, lié aux spécifications, à Xray et aux validations automatisées.
Validation	Confondre niveau d'intégration, interface, angle et type de validation.	Nommer explicitement les quatre axes.
Xray	Utiliser Xray comme archive de tests déconnectée.	Utiliser les test cases comme spécifications testables, regroupées en test sets et reliées par coverage.
Jira	Recréer toute la définition produit dans des champs Jira ou multiplier les projets/types/statuts.	Utiliser Jira pour piloter la livraison dans un même projet produit et lier le scope aux spécifications, modèles et preuves.
Qualimétrie	Mesurer ce qui est facile plutôt que ce qui aide à décider et améliorer.	Relier les métriques aux risques, preuves, critères de sortie, reprises et signaux terrain.
Automatisation / IA	Automatiser ou demander à l'IA de compenser des spécifications floues.	Utiliser l'automatisation et l'IA comme accélérateurs de comportements déjà clarifiés, traçables et validés.

Corrigé / réponses recommandées

Usage recommandé : cette section est repliée pour permettre aux participants de répondre avant de consulter les réponses. Les corrigés sont des repères de raisonnement, pas des réponses à mémoriser.

Corrigé — diagnostic initial

Question	Bonne réponse	Pourquoi
1	**B**	Le signal existe, mais il doit être qualifié avant de définir, planifier ou implémenter.
2	**B**	Une cause proposée sans preuve reste une hypothèse à documenter et valider.
3	**B**	Le RACI clarifie qui réalise, approuve, est consulté ou informé.
4	**D**	Une spécification interprétée différemment par plusieurs rôles doit être clarifiée avant d'avancer.
5	**B**	Un atelier SDLC réduit le risque par confrontation de perspectives, clarification et décision.

Corrigé — quiz bloc 1

Question	Bonne réponse	Pourquoi
1	**C**	SENSE, FORGE et CARE structurent la boucle comprendre, réaliser, observer et apprendre.
2	**C**	Passer trop vite vers l'implémentation transfère les

		ambiguïtés aux phases suivantes.
3	**D**	Un critère de sortie non satisfait doit être rendu visible et traité dans la bonne phase.

Corrigé — quiz bloc 2

Question	Bonne réponse	Pourquoi
1	**C**	Une spécification exploitable décrit une condition et un comportement attendu observable et vérifiable.
2	**D**	La formulation IF… THEN… précise le contexte, le comportement attendu et l'observable.
3	**A**	Une formulation vague doit être transformée en conditions, comportements observables, états, messages et preuves attendues.
4	**C**	La découverte de spécifications vise les comportements attendus dans les cas nominaux, limites, erreurs, dépendances et environnements.
5	**D**	Une spécification prête permet de discuter des preuves, niveaux de validation, risques et cas limites.

Corrigé — quiz bloc 3

Question	Bonne réponse	Pourquoi
1	**B**	Les quatre axes permettent de dire précisément ce qui est validé, où, avec quelle visibilité et pour quelle propriété.
2	**C**	Small, medium et large sont des niveaux d'intégration.
3	**B**	L'interface applicative est une frontière d'observation.
4	**A**	White-box, gray-box et black-box sont des angles d'observation.
5	**D**	L'intégrité des données est un type de validation.

Corrigé — quiz bloc 4

Question	Bonne réponse	Pourquoi
1	**C**	Un modèle est une représentation visuelle à aligner avec les spécifications, pas un élément optionnel ou décoratif.
2	**C**	Un mockup avec un bouton désactivé doit être relié à une spécification IF… THEN… qui précise la condition et le comportement observable.
3	**D**	La concordance signifie que le modèle, le diagramme, le

		design et les spécifications décrivent le même comportement attendu.
4	**B**	Une dépendance indisponible doit déclencher une discussion sur les cas limites, la résilience et le comportement attendu.
5	**C**	Un outil MBT peut servir de solution centrale de modélisation orientée comportement, documentation vivante et base de spécifications exécutables.
6	**D**	Le MBT génère dynamiquement des scénarios de test à partir des chemins du modèle.
7	**C**	Jira pilote la livraison, Xray porte les spécifications testables et la couverture, et les modèles comportementaux portent la logique exécutable.
8	**D**	AltWalker illustre un besoin developer-friendly : Python, CI/CD, Model Editor et intégration IDE.
9	**B**	Des outils visuels permettent aux profils PO, QA et métier de contribuer à des modèles compréhensibles tout en gardant une trajectoire MBT.

10	**A**	Le même modèle peut unir compréhension produit, documentation vivante, architecture de test, automatisation et couverture évolutive.

Corrigé — quiz bloc 5

Question	Bonne réponse	Pourquoi
1	**B**	Xray peut servir de référentiel de spécifications testables, pas seulement d'outil d'exécution.
2	**A**	Les test sets centralisent et regroupent les spécifications testables par périmètre produit.
3	**B**	Le test coverage montre quelles spécifications et validations couvrent le périmètre de livraison.
4	**B**	Jira doit piloter la livraison; la définition produit testable appartient aux spécifications et validations reliées.
5	**C**	Un inventaire figé, non maintenu et déconnecté est un anti-pattern.

Corrigé — quiz bloc 6

Question	Bonne réponse	Pourquoi

1	**C**	Un même projet conserve la traçabilité entre hiérarchie, versions, flux, dépendances, couverture et métriques.
2	**C**	La hiérarchie EPIC → FEATURE → STORY / BUG relie valeur affaire, valeur produit et réalisation concrète.
3	**C**	Les tickets orphelins perdent leur contexte produit, priorité, scope et traçabilité de validation.
4	**A**	Une relation proche de 1 FEATURE pour 1 version réduit la complexité, facilite la validation et rend la release explicable.
5	**C**	Les petites releases réduisent le risque, facilitent le diagnostic et accélèrent la confirmation terrain.
6	**A**	Le tableau produit montre le flux SDLC global; les tableaux d'équipes sont des vues opérationnelles sur les mêmes tickets.
7	**C**	L'effet Ragnarok est l'explosion de projets, types, statuts, champs, versions, tableaux et tickets orphelins.
8	**C**	Jira pilote la livraison; Xray formalise les spécifications

		testables, la couverture et les preuves.

Corrigé — quiz bloc 7

Question	Bonne réponse	Pourquoi
1	**C**	Mesurer permet d'observer si les pratiques réduisent les reprises, ambiguïtés, défauts échappés et risques résiduels.
2	**A**	Une bonne métrique aide à comprendre un risque, une décision, un apprentissage ou une amélioration du flux.
3	**C**	Automatiser des sources floues produit une ambiguïté qui s'exécute plus vite et donne une fausse impression de maîtrise.
4	**D**	Le bon moment est quand le comportement est clair, répétable, observable, à risque ou coûteux à valider manuellement.
5	**C**	L'IA peut amplifier ou reformuler proprement des ambiguïtés, incohérences et hypothèses non validées.

Corrigé — examen sommaire

Question	Bonne réponse	Pourquoi
1	**C**	Le sujet tente de passer vers PLANIFIER alors que

		DÉFINIR est incomplet.
2	**C**	Ce qui manque le plus est la stratégie de validation, les cas limites et la concordance mockup ↔ spécifications.
3	**A**	Le bon réflexe est de retourner ou rester à DÉFINIR avec actions explicites.
4	**B**	SPÉCIFICATION, Comment définir et Les tests sont les références pertinentes.
5	**C**	L'anti-pattern est de planifier alors que les critères de définition ne sont pas satisfaits.

Corrigé — examen récapitulatif

Question	Bonne réponse	Pourquoi
1	**C**	Le signal, la source, l'impact, les faits et les hypothèses RCA doivent être qualifiés en premier.
2	**C**	Les options techniques ne suffisent pas : comportements, impacts, risques, modèles et preuves manquent encore.
3	**B**	DÉFINIR doit produire des spécifications observables, modèles concordants, cas limites, impacts, risques et preuves attendues.

4	**A**	PLANIFIER devient pertinent quand les critères de sortie de DÉFINIR sont suffisamment satisfaits et que la DoR peut être évaluée.
5	**C**	Une trace exploitable contient décision, justification, risques, actions, owners, références, spécifications impactées, modèles liés et prochaine étape.

Étape 10 — Passage vers la pratique

Une fois la formation théorique complétée, les participants doivent appliquer les acquis dans un atelier réel à l'aide de la page Pratique atelier — Le cycle complet de la réalisation du produit.

La formation répond au **pourquoi** et au **quoi maîtriser**. La page pratique sert à documenter le **comment appliquer** dans un contexte réel : contexte d'atelier, participants, décisions, ambiguïtés, risques, actions, owners, liens Jira/Xray, modèles, preuves et suivis.

Contrôle qualité de la formation : le parcours couvre maintenant les étapes 0 à 10, les quiz blocs 1 à 7, l'examen sommaire, l'examen récapitulatif et les corrigés associés. Les technologies citées sont présentées comme des exemples de moyens; l'objectif demeure la fondation minimale viable pour standardiser la livraison, documenter le produit par des spécifications fiables et relier spécifications, modèles, preuves, automatisation et IA.